Segunda Entrega del Proyecto
Ingeniería de Software II

Presentado por:
-Samuel Giraldo Pavas
-Carlos Francisco Moreno Casas

22 de Octubre de 2026



Universidad Nacional de Colombia
Sede Manizales
Facultad de Administración

Diseño y Arquitectura

Contextualización de lo antes planteado

Introducción y Alcance

- El sistema estará orientado al control de inventarios de una empresa de mensajería, principalmente de los insumos necesarios para sus operaciones, como cajas, sobres, bolsas de seguridad, cinta de embalaje y etiquetas

- Este permitirá administrar los productos y sus categorías, los proveedores, las órdenes de compra, las existencias por centro de distribución y los movimientos de entrada, salida, ajuste y traslado, este contempla alertas de stock mínimo, trazabilidad de las operaciones y reportes de inventario.

- Como función complementaria se incluye la gestión de envíos permitiendo relacionar el uso de insumos con las actividades logísticas. Los paquetes entregados como clientes se distinguen del inventario propio de la empresa puesto que corresponden a mercancías bajo custodia para su próximo transporte.

Caso Ilustrativo

- Trabajaremos con base en el siguiente caso:
    1. La empresa registra el producto “Caja mediana”
    2. Registra un proveedor de materiales de embalaje
    3. Genera una orden de compra de 100 cajas
    4. Al recibirlas, registra una entrada de 100 unidades en una bodega
    5. Para preparar un envío se utilizan 2 cajas
    6. Registra una salida por consumo y quedan 98 unidades
    7. Conserva quien realizó el movimiento, cuando y para que envío se utilizó

Diagramas

Justificaciones de lo graficado

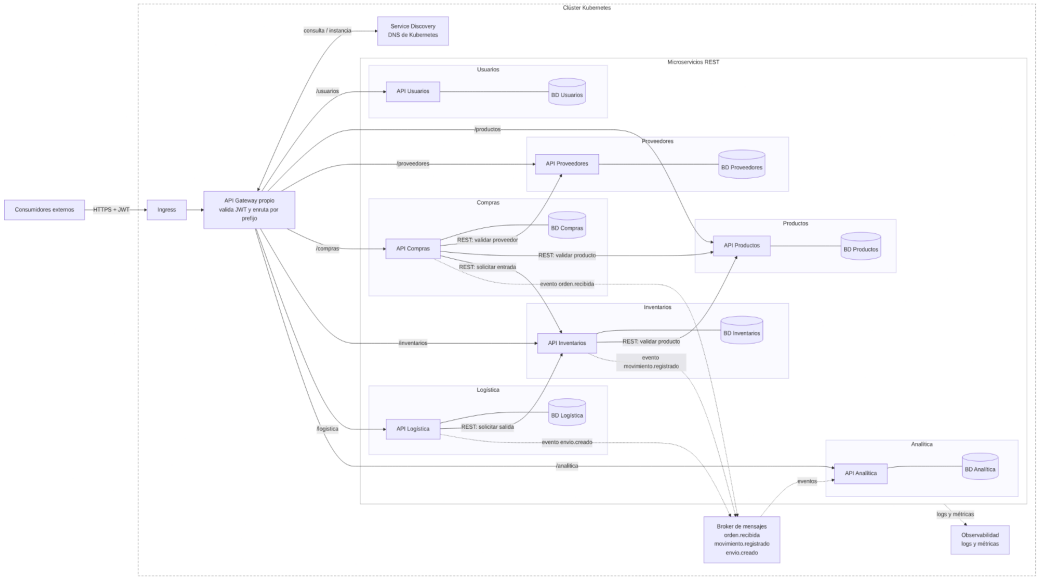
Arquitectura de Microservicios

- El sistema de control de inventarios para una empresa de mensajería se diseñará mediante una arquitectura de microservicios, organizada en componentes con responsabilidades delimitadas. Su objetivo principal será gestionar los insumos utilizados para las operaciones de la empresa como cajas, sobres, bolsas de seguridad, cinta de embalaje y etiquetas. La solución estará compuesta por siete microservicios: Productos, Proveedores, Compras, Inventarios, Usuarios, Logistica y Analitica, cada servicio administra su propia información y expondrá sus funcionalidades mediante una API.
- Los paquetes entregados por los clientes se gestionan desde Logística y se distinguen del inventario propio de los insumos de la empresa, de esta manera el sistema podrá relacional el consumo de materiales con los envíos sin confundir ambos tipos de elementos
- Las solicitudes externas ingresarán mediante una API Gateway desarrollado por el equipo, la comunicación combinara operaciones sincronizadas mediante API REST y eventos asíncronos mediante broker de mensajes, el despliegue se planificará sobre Kubernetes utilizando su mecanismo de descubrimiento de servicios.

Diagrama General de la Arquitectura

- El diagrama presenta los componentes del sistema y sus relaciones principales. Los consumidores externos acceden a las API mediante el Gateway que dirigira las solicitudes al microservicio que le corresponda a la operación
- Cada microservicio contará con una base de datos independiente, el descubrimiento de servicios permitirá localizar los destinos internos por nombre.
- Una petición recorre el sistema así: el consumidor externo, ya sea la aplicación web o móvil, envía la solicitud por HTTPS con su token JWT. Esta entra al clúster Kubernetes y llega al API Gateway propio. El Gateway verifica el token, identifica el microservicio destino por el prefijo de la ruta (por ejemplo, /inventarios) y consulta al Service Discovery, que en Kubernetes es su DNS interno, para obtener la dirección de una instancia disponible de ese servicio. Luego reenvía la petición junto con la identidad del usuario y devuelve la respuesta al consumidor. Si para responder un servicio necesita algo de otro, lo llama directamente por REST usando su nombre interno, sin pasar por el Gateway; por ejemplo, Compras llama a Inventarios para solicitar una entrada. Los hechos que otros servicios procesan después (una orden recibida, un movimiento registrado, un envío creado) se publican como eventos en el broker de mensajes, que Analítica consume. Todos los servicios envían sus logs y métricas al sistema de observabilidad.

Diagrama 1. Arquitectura general de microservicios



Microservicios y responsabilidades

- La separación de responsabilidades busca que cada servicio sea propietario de un conjunto definido de datos y operaciones con los mismos, cuando un servicio necesite información o acciones de otro utilizara su API sin acceder directamente a su base de datos.

Microservicio	***Responsabilidad***	***Información a su Cargo***	***Límite***
Productos	Administra el catálogo de insumos	Productos, categorias, subcategorias, unidades de medida y datos descriptivos	No administra la cantidad disponible en bodega
Proveedores	Administrar los proveedores de insumos	Identificación, contacto y condiciones comerciales	No modifica existencias ni recibe compras
Compras	Gestionar el abastecimiento	Órdenes de compra, detalles, costos y recepciones.	Solicita las entradas a Inventarios y no actualiza directamente su base de datos.
Inventarios	Controla existencias y movimientos	Bodegas, saldos, entradas, salidas, ajustes, traslados,	No administra guías ni estados de entrega

		devoluciones y stock.	
Usuarios	Gestionar el acceso al sistema	Usuarios, roles, permisos y datos de autenticación	No administra las operaciones de los otros dominios
Logística	Gestiona envíos y el uso de insumos asociados.	Clientes, paquetes, guías y estados de envío	No modifica directamente las existencias
Analitica	Elabora reportes e indicadores	Resúmenes e información derivada para consultas.	No es la fuente autorizada para modificar el inventario

API Gateway Propio

- El API Gateway propio será el punto de entrada de las solicitudes que se le realicen al programa dirigidas a la API del sistema, su responsabilidad principal será identificar el destino de cada petición, enviarla al microservicio correspondiente y devolver su respuesta al consumidor, éste será desarrollado sin reglas del negocio de compras, inventarios o logística debido a que estas se encontraran en sus respectivos servicios.
- Este válida el acceso a las rutas protegidas de los microservicios, las rutas propuestas son las siguientes:
    - /productos
    - /proveedores
    - /compras
    - /inventarios
    - /usuarios
    - /logistica
    - /analitica

Diseño de Contratos de API por Microservicio (OpenAPI/Swagger)

Todas las solicitudes externas ingresan por el API Gateway propio, que dirige cada petición al microservicio correspondiente según el prefijo de la ruta. Las rutas protegidas requieren un token JWT en el encabezado Authorization (Bearer). Los errores siguen un formato común con código, mensaje y detalles.

Productos (/productos)

- GET /productos: lista productos con filtros por categoría y búsqueda, paginada.
- POST /productos: crea un producto (201). Responde 409 si el SKU ya existe.
- GET /productos/{id} y PUT /productos/{id}: consulta y actualiza un producto.
- GET y POST /productos/categorias: lista y crea categorías y subcategorías.
- GET /productos/unidades-medida: lista las unidades de medida.

Proveedores (/proveedores)

- GET y POST /proveedores: lista y registra proveedores. Responde 409 si el NIT ya existe.
- GET y PUT /proveedores/{id}: consulta y actualiza un proveedor.
- PATCH /proveedores/{id}/estado: activa o desactiva un proveedor.

Compras (/compras)

- GET y POST /compras/ordenes: lista y crea órdenes de compra en estado BORRADOR.
- GET /compras/ordenes/{id}: consulta la orden con sus detalles y recepciones.
- POST /compras/ordenes/{id}/emitir: pasa la orden de BORRADOR a EMITIDA. Responde 409 si el estado no lo permite.
- POST /compras/ordenes/{id}/cancelar: cancela la orden solo si no tiene recepciones.
- POST /compras/ordenes/{id}/recepciones: registra la recepción y solicita a Inventarios la entrada de existencias. Responde 502 si Inventarios no está disponible.

Inventarios (/inventarios)

- GET y POST /inventarios/bodegas: lista y crea bodegas o centros de distribución.
- GET /inventarios/saldos: consulta existencias por producto y bodega.
- PUT /inventarios/saldos/{producto_id}/{bodega_id}/stock-minimo: define el stock mínimo.
- GET /inventarios/alertas/stock-minimo: lista los productos por debajo del mínimo.
- GET /inventarios/movimientos: consulta movimientos con filtros por producto, bodega, tipo, referencia y fechas.
- POST /inventarios/movimientos: registra una ENTRADA, SALIDA, AJUSTE, TRASLADO o DEVOLUCIÓN. Responde 409 con STOCK_INSUFICIENTE cuando no hay existencias suficientes.
- GET /inventarios/movimientos/{id}: consulta un movimiento.

Usuarios (/usuarios)

- POST /usuarios/auth/login: autentica y devuelve el token JWT.
- GET /usuarios/auth/validar: valida un token (uso interno del Gateway).
- GET y POST /usuarios: lista y crea usuarios.
- GET y PUT /usuarios/{id}: consulta y actualiza un usuario.
- PUT /usuarios/{id}/roles: reemplaza los roles del usuario.
- GET /usuarios/roles y GET /usuarios/permisos: lista roles y permisos.

Logística (/logistica)

- GET y POST /logistica/clientes: lista y registra clientes.
- GET y POST /logistica/paquetes: lista y registra paquetes en custodia.
- GET y POST /logistica/envios: lista y crea envíos con su guía.
- GET /logistica/envios/{id}: consulta el envío con paquetes, insumos e historial de estados.
- POST /logistica/envios/{id}/estado: cambia el estado del envío.
- POST /logistica/envios/{id}/insumos: registra el consumo de insumos. Por cada línea se genera una SALIDA en Inventarios con referencia al envío.

Analítica (/analitica), solo lectura

- GET /analitica/indicadores: lista los indicadores y su último valor.
- GET /analitica/indicadores/{nombre}: serie histórica de un indicador.
- GET /analitica/reportes/inventario: resumen de inventario por bodega, en JSON o CSV.

Modelo de Datos por Microservicio (Diagramas Entidad-Relación)

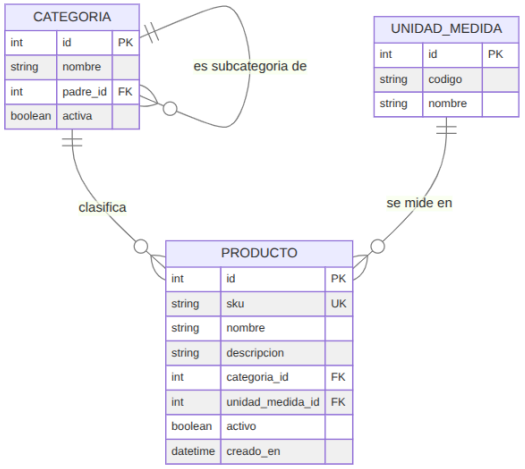
Cada microservicio tiene su propia base de datos. Las referencias a otros servicios son identificadores (por ejemplo producto_id o bodega_id), no claves foráneas físicas.

Productos

- CATEGORIA (id, nombre, padre_id, activa). Las subcategorías se relacionan con la categoría padre mediante padre_id.
- UNIDAD_MEDIDA (id, codigo, nombre).
-PRODUCTO (id, sku [único], nombre, descripcion, categoria_id, unidad_medida_id, activo, creado_en)

Relaciones: una categoría clasifica muchos productos; una unidad de medida aplica a muchos productos.

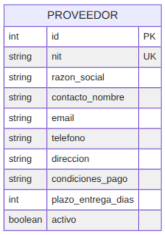
Diagrama 2. Modelo de datos: Productos



Proveedores

- PROVEEDOR (id, nit [único], razon_social, contacto_nombre, email, telefono, direccion, condiciones_pago, plazo_entrega_dias, activo).

Diagrama 3. Modelo de datos: Proveedores

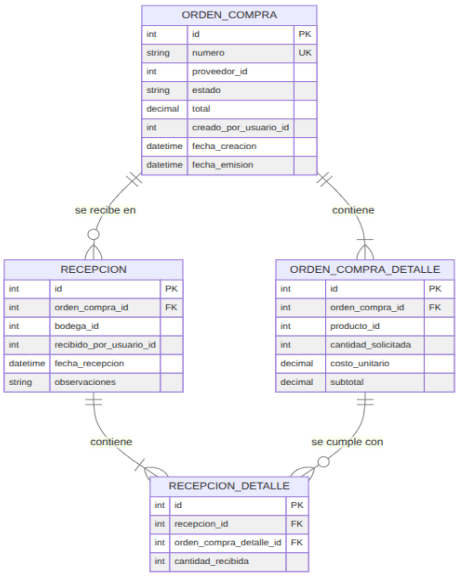


Compras

- ORDEN_COMPRA (id, numero [único], proveedor_id, estado, total, creado_por_usuario_id, fecha_creacion, fecha_emision).
- ORDEN_COMPRA_DETALLE (id, orden_compra_id, producto_id, cantidad_solicitada, costo_unitario, subtotal).

- RECEPCION (id, orden_compra_id, bodega_id, recibido_por_usuario_id, fecha_recepcion, observaciones).
- RECEPCION_DETALLE (id, recepcion_id, orden_compra_detalle_id, cantidad_recibida).
Relaciones: una orden tiene muchos detalles y puede tener varias recepciones; cada línea de recepción cumple una línea de la orden.

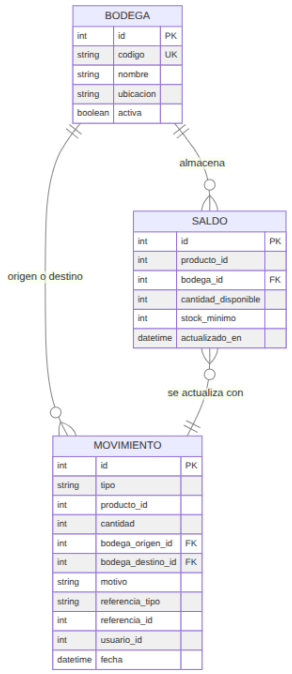
Diagrama 4. Modelo de datos: Compras



Inventarios

- BODEGA (id, codigo [único], nombre, ubicacion, activa).
- SALDO (id, producto_id, bodega_id, cantidad_disponible, stock_minimo, actualizado_en). La combinación producto_id y bodega_id es única.
- MOVIMIENTO (id, tipo, producto_id, cantidad, bodega_origen_id, bodega_destino_id, motivo, referencia_tipo, referencia_id, usuario_id, fecha).
Relaciones: una bodega almacena muchos saldos y participa en muchos movimientos; cada movimiento actualiza un saldo.

Diagrama 5. Modelo de datos: Inventarios

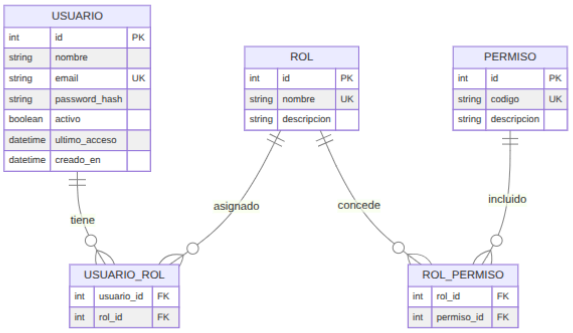


Usuarios

- USUARIO (id, nombre, email [único], password_hash, activo, ultimo_acceso, creado_en).
- ROL (id, nombre [único], descripcion).
- PERMISO (id, codigo [único], descripcion).
- USUARIO_ROL y ROL_PERMISO son tablas de relación de muchos a muchos.

Relaciones: un usuario puede tener varios roles y un rol puede asignarse a varios usuarios; un rol concede varios permisos y un permiso puede estar incluido en varios roles.

Diagrama 6. Modelo de datos: Usuarios



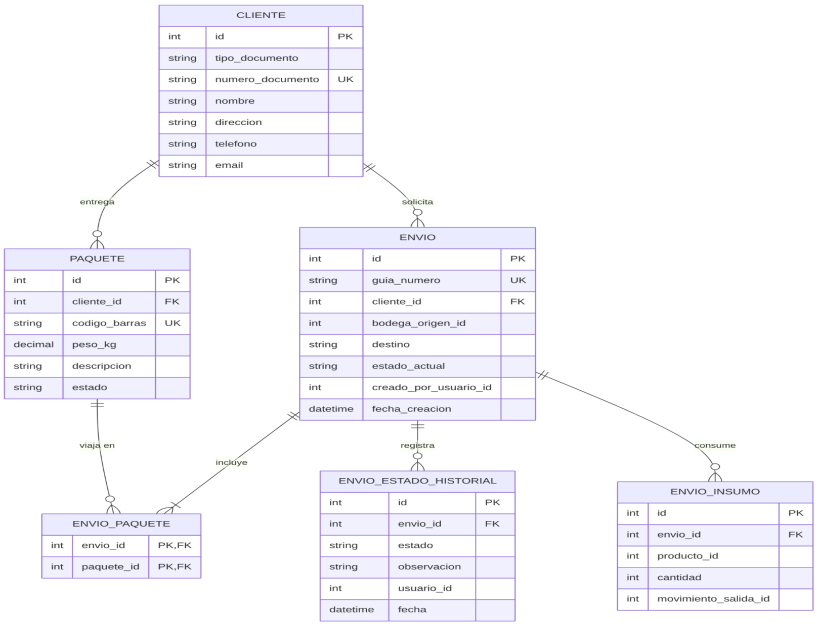
Logística

- CLIENTE (id, tipo_documento, numero_documento [único], nombre, direccion, telefono, email).
- PAQUETE (id, cliente_id, codigo_barras [único], peso_kg, descripcion, estado).

- ENVIO (id, guia_numero [único], cliente_id, bodega_origen_id, destino, estado_actual, creado_por_usuario_id, fecha_creacion).
- ENVIO_PAQUETE (envio_id, paquete_id).
- ENVIO_ESTADO_HISTORIAL (id, envio_id, estado, observacion, usuario_id, fecha).
- ENVIO_INSUMO (id, envio_id, producto_id, cantidad, movimiento_salida_id).

Relaciones: un cliente solicita muchos envíos y entrega muchos paquetes; un envío incluye varios paquetes y consume varios insumos. Cada consumo guarda el identificador del movimiento de salida en Inventarios.

Diagrama 7. Modelo de datos: Logística

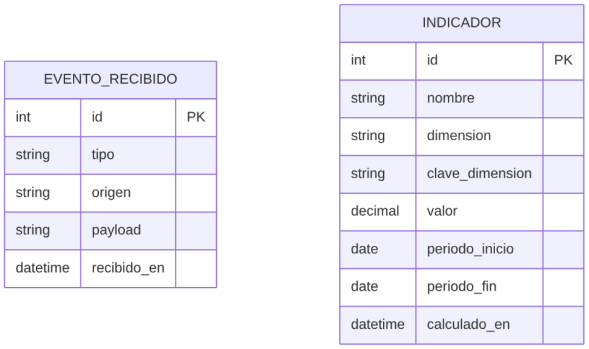


Analítica

- EVENTO_RECIBIDO (id, tipo, origen, payload, recibido_en): conserva los eventos que recibe del broker (orden.recibida, movimiento.registrado y envio.creado) con su contenido en formato JSON.
-INDICADOR (id, nombre, dimension, clave_dimension, valor, periodo_inicio, periodo_fin, calculado_en): valores calculados a partir de los eventos almacenados, como existencias o rotación por bodega.

Relaciones: las dos tablas no se relacionan entre sí; los indicadores se calculan a partir de los eventos guardados. Analítica no administra tablas de negocio de otros servicios ni modifica su información: sólo conserva los eventos y deriva los indicadores.

Diagrama 8. Modelo de datos: Analítica



Flujo del caso ilustrativo sobre los modelos:

1. Productos crea el producto "Caja mediana".
2. Proveedores registra el proveedor de embalaje.
3. Compras crea y emite la orden de 100 cajas.
4. La recepción genera en Inventarios una ENTRADA de 100 unidades con referencia a la orden de compra. Saldo: 100.
5. Logística crea el envío y registra el consumo. Inventarios genera una SALIDA de 2 unidades con referencia al envío. Saldo: 98.
6. La salida conserva el usuario, la fecha y el envío asociado.